

Trabajo Práctico Final – I-402

SLAM, Navegación Autónoma y Despliegue en Hardware Real

Francesca Ragonesi Romanella Colombini Eliana Ostrovsky Zoë Velazquez Zorzi

Universidad de San Andrés – Departamento de Ingeniería

4 de julio de 2026

Resumen. Sistema robótico autónomo completo sobre ROS 2 Humble para TurtleBot4. La **Parte A** implementa *Graph SLAM* con cierre de lazo por marcadores ArUco sobre datos reales pregrabados, produciendo un mapa de grilla de ocupación (0,05 m/celda). La **Parte B** combina un filtro de partículas (MCL, $N = 200$), planificación *any-angle* con Theta*, seguimiento con Pure Pursuit y una máquina de estados con evasión reactiva de obstáculos no mapeados. La **Parte C** extiende la plataforma al TurtleBot4 físico, sumando detección de conos rojos por segmentación HSV fusionada con LIDAR.

1. Introducción

Elegimos la **Opción 3: Features con Cámara**, que trabaja sobre datos reales pregrabados del TurtleBot4 en lugar de simulación. Esto obliga a enfrentar efectos ópticos reales (desenfoque, baja densidad de marcadores, ausencia de *ground truth*) y permite reutilizar el mismo mapa en el robot físico de la Parte C. El flujo de trabajo siguió la cadena:

RosBag $\rightarrow$ Graph SLAM $\rightarrow$ MCL + Theta* + PP $\rightarrow$ TB4 real

La validación fue incremental: primero contra bags pregrabados y luego en el robot físico, donde el tiempo de acceso al hardware es acotado.

2. Parte A: Percepción y SLAM

2.1. Arquitectura

La solución se organizó en dos paquetes. `aruco_pkg` concentra la percepción y la odometría: `aruco_detector_node` detecta marcadores y estima su pose 3D con `solvePnP`, mientras que `odom_delta_node` computa el modelo de deltas sobre `/tb4_0/odom`. `slam_pkg` contiene el núcleo de Graph SLAM (`graph_slam.py`), la grilla de ocupación y el *pipeline* completo (`slam_pipeline.py`).

```
python3 slam_pipeline.py \
--odom odom_deltas.csv \
--aruco laberinto_detections.csv \
--scans scans.csv \
--noise-model noise_model.json \
--sm-passes 3 --max-scans 19926 \
--min-hits-occ 500
```

2.2. Modelo de odometría

Los deltas ($\delta_{rot1}, \delta_{trans}, \delta_{rot2}$) se calculan entre pares de poses consecutivas de `/tb4_0/odom`. Como el bag publica con QoS `BEST_EFFORT` pero ROS 2 suscribe con `RELIABLE` por defecto, el desajuste resulta en cero mensajes entregados sin error explícito, por lo cual declarar `BEST_EFFORT` explícitamente lo resolvió. Lo anterior se aplicó a todos los sensores del TB4.

2.3. Detección de marcadores ArUco

El diccionario ArUco fue el principal obstáculo. Los marcadores reales son `DICT_4X4_50` pero la configuración inicial era `DICT_5X5_250`: con el diccionario equivocado el detector encontraba candidatos pero decodificaba cero IDs, sin lanzar ninguna excepción.

Diagnóstico: el script `aruco_dict_probe.py` prueba por fuerza bruta una veintena de diccionarios sobre pocos frames. Al corregirlo se pasó de 1 detección en todo el bag a cientos.

El tópic de imagen es un *preview* de baja resolución (OAK-D), por lo que el nodo agranda la imagen $4\times$ con interpolación Lanczos antes de detectar, escalando K de forma consistente. El ruido se modela como $\sigma = a + b d$ por eje (calibrado con el bag `aruco_estimation`) y propagado a rango-bearing mediante el Jacobiano.

2.4. Algoritmo de Graph SLAM

El optimizador es **Levenberg–Marquardt** disperso sobre un grafo de poses y landmarks: se probó Gauss-Newton puro, pero en el grafo grande divergía (las poses escapaban a escala de kilómetros), y el *damping* de LM sumado a aceptar solo los pasos que reducen el error lo estabiliza. Se minimiza:

$$F(\mathbf{x}) = \sum_{\text{odom.}} \mathbf{e}_{ij}^\top \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij} + \sum_{\text{ArUco}} \mathbf{e}_{ik}^\top \boldsymbol{\Omega}_{ik} \mathbf{e}_{ik}$$

En cada iteración se ensambla el sistema normal $(\mathbf{H} + \lambda \mathbf{I})\Delta \mathbf{x} = -\mathbf{b}$ con $\mathbf{H} = \sum \mathbf{J}^\top \boldsymbol{\Omega} \mathbf{J}$, donde λ es el factor de *damping* de Levenberg–Marquardt, y se resuelve el incremento. Los ángulos se normalizan a $[-\pi, \pi]$ en cada residuo.

El **cierre de lazo** emerge naturalmente: el ID de cada ArUco actúa como asociación de datos trivial, de modo

que al visitar un landmark la restricción adicional corrige la deriva acumulada. Las $\approx 49k$ detecciones se agregan por par (keyframe, landmark) —promediando rango y *bearing*— reduciendo el sistema a $\approx 1,1k$ aristas de observación bien condicionadas. La trayectoria se submuestra a *keyframes* (≈ 2200 en la corrida final) para escalar a los 46,8 minutos del bag.

2.5. Segunda pasada: grilla de ocupación

Con la trayectoria optimizada, cada barrido de LIDAR proyecta los rayos sobre las poses corregidas: Bresenham marca celdas libres y la celda de impacto se acumula en *log-odds*. Antes de rasterizar se corrige la extrínseca del LIDAR —el rplidar del TB4 está girado $+90^\circ$ respecto de *base_link*, sin lo cual las paredes rotan al girar el robot y el mapa se emborrona— y se descartan los retornos más allá de ≈ 5 m (*chorros* espurios; el 99 % cae bajo 3,2 m). Un refinador de *scan-matching* estilo Hector SLAM (campo de verosimilitud + Gauss-Newton, *coarse-to-fine*, 3 pasadas) adelgaza el espesor de las paredes.

Bug – paredes internas invisibles: el *clip* del log-odds (*clamp* = 5) se aplicaba en *cada* actualización, no solo al final. En un laberinto recorrido en múltiples lazos una misma celda se observa miles de veces, así que el mapa terminaba reflejando solo los últimos ≈ 6 –12 eventos en vez de la mayoría histórica: una celda con 7 784 impactos reales quedaba saturada como “libre” y las paredes internas desaparecían. Subir el *clamp* a 50 las recupera. Un rescate por evidencia absoluta (*--min-hits-occ*) exporta como ocupadas las celdas con muchos impactos crudos que el consenso borraría (patas de silla de ≈ 2 cm que no llenan la celda de 5 cm).

Parámetro	Valor
Resolución	0,05 m/celda
Landmarks	46
Keyframes	≈ 2200
sm-passes	3
min-hits-occ	500
clamp (log-odds)	50
χ^2 (inicial $\rightarrow$ final)	54 442 $\rightarrow$ 1 686

Cuadro 1: Parámetros del mapa generado (corrida larga, 46,8 min).

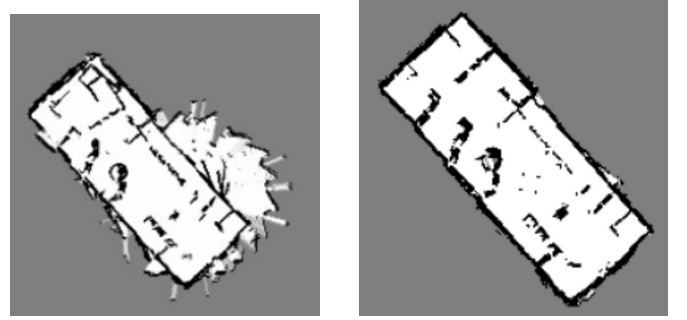
2.6. Resultados del mapeo

Las primeras versiones del mapa mostraban ruido considerable, superposición de paredes y celdas libres marcadas erróneamente (Figura 1a). Hubo tres causas encadenadas: observaciones de ArUco espurias que metían restricciones erróneas en el grafo, la extrínseca del LIDAR sin corregir, y el *clip* del log-odds que borraba las paredes internas.

El **gating por χ^2** (distancia de Mahalanobis) descarta las observaciones más ruidosas y re-optimiza; sobre la corrida final el χ^2 cae de 54 442 a 1 686. El **scan-matching** *coarse-to-fine* (3 pasadas) adelgaza y alinea las paredes,

y el ajuste del *clamp* recupera la estructura interna del laberinto. El resultado es el mapa definitivo de la Figura 1b.

Limitación: las zonas con mucha rotación del robot o con cobertura escasa de ArUcos quedan algo más sucias: sin referencias visuales suficientes la deriva de odometría se corrige menos. Se optó por documentar esta limitación en lugar de sobre-ajustar hiperparámetros a este bag puntual.



(a) Mapa intermedio con ruido.

(b) Mapa definitivo.

Figura 1: Evolución del mapa durante el ajuste de hiperparámetros.

3. Parte B: Navegación Autónoma

3.1. Arquitectura general

La Parte B encadena cuatro módulos: filtro de partículas (MCL), planificador Theta*, controlador Pure Pursuit y una máquina de estados que orquesta el flujo. El lanzamiento requiere dos terminales: el bag (para pruebas sin robot) y el launch completo:

```
ros2 bag play ... --loop \
--remap /tb4_0/tf:=/tf \
/tb4_0/tf_static:=/tf_static
ros2 launch tpf robot_real.launch.py
```

3.2. Localización: Filtro de Partículas (MCL)

El nodo `localization_node` mantiene $N = 200$ partículas, cada una con hipótesis de pose (x, y, θ) en el frame del mapa.

Al arrancar, el robot no conoce su pose y permanece en `WAITING`. La herramienta *2D Pose Estimate* de RViz publica en `/initialpose` un `PoseWithCovarianceStamped`; en ese callback se distribuyen todas las partículas alrededor de la pose indicada con ruido gaussiano ($\sigma = 0,05$ en x , y y θ), conservando dispersión para absorber el error inicial.

Con cada mensaje de odometría se calcula el desplazamiento relativo mediante el modelo de deltas $(\delta_{rot1}, \delta_{trans}, \delta_{rot2})$, normalizando ángulos a $[-\pi, \pi]$. Cada partícula se propaga inyectando ruido dependiente de

la magnitud del movimiento:

$$\hat{\delta}_{rot1} = \delta_{rot1} + \mathcal{N}(0, \alpha_1 |\delta_{rot1}| + \alpha_2 \delta_{trans}), \quad (1)$$

$$\hat{\delta}_{trans} = \delta_{trans} + \mathcal{N}(0, \alpha_3 \delta_{trans} + \alpha_4 (\dots)), \quad (2)$$

$$\hat{\delta}_{rot2} = \delta_{rot2} + \mathcal{N}(0, \alpha_1 |\delta_{rot2}| + \alpha_2 \delta_{trans}). \quad (3)$$

El nodo `likelihood_field` precomputa, para cada celda libre, la distancia a la pared más cercana (transformada de distancia euclídea) y la convierte en verosimilitud $p = \exp(-d^2/2\sigma^2)$ con $\sigma = 0,35$ m. Para cada partícula, los rayos válidos del LIDAR se proyectan a coordenadas globales según su pose hipótesis y se consultan en el campo; el peso se acumula en dominio logarítmico para estabilidad numérica. En el robot real se descartan rayos con intensidad nula y se compensa el offset angular de 90° del LIDAR del TB4.

Los pesos se normalizan y las partículas se remuestran con *Stochastic Universal Sampling* (SUS), lo que reduce la varianza respecto del muestreo multinomial. La pose estimada se obtiene como promedio ponderado (orientación promediada en el círculo unitario) y se publica en `/estimated_pose`; la nube completa va a `/belief`.

Métricas de convergencia.

$$\text{spread}_{xy} = \text{Var}(x_i) + \text{Var}(y_i), \quad \text{spread}_\theta = 1 - \left| \frac{1}{N} \sum e^{j\theta_i} \right|.$$

El robot permanece en `WAITING` hasta que ambas caen bajo los umbrales ($0,25 \text{ m}^2$ y $0,10$). Si se elevan durante el recorrido, el robot se detiene hasta re-converger.

Bug: TF tree incompleto. `localization_node.py` declaraba un `TransformBroadcaster` pero nunca llamaba a `sendTransform()`. El frame `map` no existía en el árbol TF y las partículas no aparecían en RViz. Se tuvo que implementar y llamar `_broadcast_map_to_odom()`.

3.3. Inflado del mapa

El mapa se dilata con una máscara circular de radio 4 celdas ($0,20$ m) con `binary_dilation` de SciPy. Las celdas desconocidas (-1) se preservan; solo las libres (0) dentro del radio pasan a ocupadas (100).

3.4. Planificación: Theta*

Theta* permite conexiones directas entre nodos no adyacentes cuando existe línea de visin libre, produciendo caminos *any-angle* más cortos y naturales que la 8-vecindad clásica. La línea de visión se verifica con Bresenham: si alguna celda está ocupada (≥ 50) o desconocida (-1), se descarta.

Antes de cada planeo se construye un *planning map* temporal que copia del mapa inflado estático y le agrega los obstáculos del último scan proyectados e inflados.

3.5. Seguimiento: Pure Pursuit

Con distancia de anticipación $L = 0,2$ m y velocidad lineal constante $v = 0,1$ m/s, en cada ciclo se descarta todo

waypoint dentro de L , se toma el primero restante a distancia $\geq L$ y se calcula:

$$\alpha = \text{atan2}(\sin(\phi - \theta), \cos(\phi - \theta)), \quad \omega = \frac{2v \sin \alpha}{L}.$$

La velocidad lineal constante con ω proporcional a $\sin \alpha$ produce seguimiento suave sin giros bruscos. Al llegar a la posición del objetivo (tolerancia $0,10$ m), la FSM pasa a `ALIGNING`: el robot gira en el lugar hacia el yaw del goal ($0,3$ rad/s) hasta reducir el error por debajo de 12° .

3.6. Obstáculos dinámicos y re-planificación

Durante `WALKING` se evalúa un cono frontal de $\pm 45^\circ$ del LIDAR. Si alguna lectura cae a menos de $0,20$ m y la celda correspondiente en el mapa inflado está libre, se activa la transición a `AVOIDING`.

En `AVOIDING` el robot compara la distancia mínima libre a izquierda y derecha ($\pm 90^\circ$) y gira 80° hacia el lado más despejado. Completada la maniobra, espera a que el filtro de partículas reconverja y luego re-planifica desde la nueva orientación. Un cooldown de 1 s evita oscilaciones ante el mismo obstáculo.

Un nuevo goal recibido en cualquier momento tiene prioridad máxima: interrumpe `WALKING` y `ALIGNING` para volver a `PLANNING`.

3.7. Máquina de estados (Parte B)

Antes de ejecutar cualquier estado se verifica que la localización sea confiable (umbrales de convergencia). Si la dispersión supera el límite de degradación, el robot se detiene en su estado actual hasta re-converger.

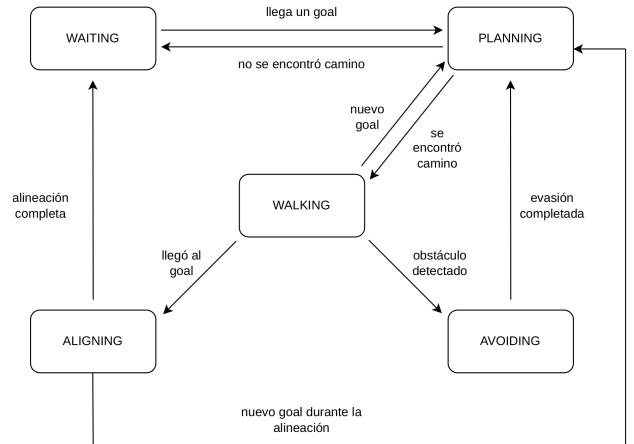


Figura 2: Diagrama de bloques de la máquina de estados del nodo de navegación.

3.8. Bugs y problemas durante la integración

La mayoría de los problemas aparecen cuando todos los nodos interactúan en tiempo real, no en pruebas aisladas.

QoS mismatch (cero mensajes sin error): El TB4 publica sensores con BEST_EFFORT, pero los nodos usan RELIABLE por defecto. El suscriptor queda en silencio, no se ninguna excepción, solo un *warning* de incompatibilidad de QoS en los logs. La solución fue declarar BEST_EFFORT explícitamente en todos los suscriptores de sensores.

Robot encerrado al planificar: Durante el día de prueba, se notó que, debido al ruido del LIDAR, el inflado que utilizaba sensado para objetos dinámicos encerraba al robot y afectaba la planificación. Como solución se utilizaron dos métodos para tratar el ruido: El primero consiste de observar un radio de vecindad bajo la suposición de que un objeto dinámico genera un cluster de puntos más que puntos individuales esporádicos. El segundo consiste en utilizar la información de 5 scans diferentes para no considerar aquellos puntos que no aparecen en el mismo lugar en el tiempo.

3.9. Resultados

El filtro de partículas localizó al robot establemente tras el *2D Pose Estimate*. Theta* generó rutas que respetan el margen del mapa inflado, y Pure Pursuit las siguió sin oscilaciones. Se verificó el re-planeo ante nuevos goals y la evasión reactiva ante obstáculos. La Figura 3 muestra RViz durante una prueba real.

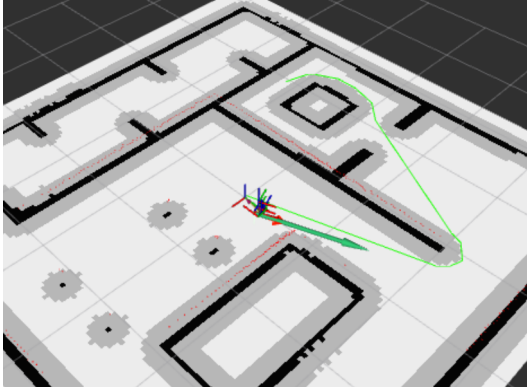


Figura 3: Navegación autónoma – Parte B. Vista general del mapa y trayectoria planificada.

4. Parte C: Despliegue en Hardware Real

4.1. Portado al TurtleBot4 físico

El portado reutiliza los módulos de la Parte B con adaptaciones puntuales: topics a `/tb4_0/*`, BEST_EFFORT en todos los suscriptores de sensores, offset LIDAR de 90° ($\theta_{robot} = \theta_{lidar} + 90^\circ$ en todos los métodos que iteran sobre el scan), filtro de intensidades (`intensities[i]==0` se descarta) y `TimerAction` de 3s en el launch file para que el hardware se inicialice.

```
ros2 launch tpf robot_real.launch.py # T1
ros2 run cono_detector_pkg detector_cono # T2
```

4.2. Detección de conos rojos

El nodo `detector_cono_node` segmenta en HSV. El rojo cruza la discontinuidad del tono en 0/180 de OpenCV, por lo que se usan dos rangos combinados con OR:

Rango	Hue	Sat.	Val.
Inferior	[0, 10]	[120, 255]	[70, 255]
Superior	[170, 180]	[120, 255]	[70, 255]

Cuadro 2: Umbrales HSV para detección de cono rojo.

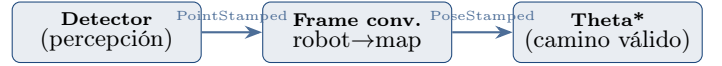
Si el contorno rojo supera 300 px², se estima el *bearing*:

$$\alpha_{cono} = \arctan\left(\frac{c_{x,cono} - c_x}{f_x}\right)$$

La distancia se obtiene indexando el scan LIDAR en esa dirección (ventana ± 3 rayos). El resultado se publica como `PointStamped` en el frame del robot, junto con una imagen de *debug* para ajustar umbrales en el lab.

4.3. Validación geométrica (paredes caladas)

Dado que ver un cono a través de una abertura no implica que el camino directo esté libre, el diseño separa percepción de planificación:



La percepción propone el destino; el planificador garantiza el camino. Esta arquitectura corrigió un bug de diseño: un estado `APPROACHING_CONE` con P-controller directo enviaba al robot contra las paredes. Al eliminarlo y delegar a Theta*, el comportamiento se volvió correcto por construcción.

4.4. Máquina de estados (Parte C)

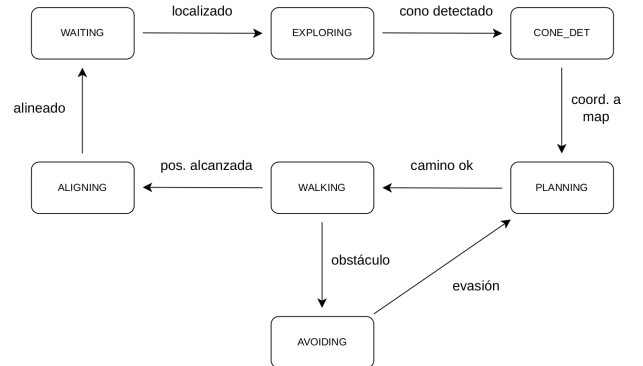


Figura 4: Diagrama de bloques de la máquina de estados del nodo de exploración de cono.

EXPLORING: El robot avanza y gira hacia el lado más despejado (reactivo, según LIDAR) buscando conos. La transición a `CONE_DETECTED` se dispara cuando el detector reporta un contorno rojo de área suficiente.

CONE_DETECTED: La coordenada relativa se transforma al frame del mapa usando la pose estimada actual. El goal se coloca a 0,40m antes del cono para no apuntar a la pared donde está apoyado. El ciclo de Parte B se encarga del resto.

4.5. Resultados

La validación se realizó contra el bag de visión de la cámara. El detector reportó correctamente bearing y distancia, la FSM transitó a **CONE_DETECTED**, Θ^* trazó la ruta y Pure Pursuit ejecutó el seguimiento. La Figura 5 muestra la detección y la trayectoria planificada.



Figura 5

5. Hiperparámetros del sistema

Parámetro	Valor
Resolución del mapa	0,05 m/celda
gate-chi2 (SLAM)	2,0
sm-passes	6
Partículas MCL	200
σ likelihood field	0,35 m
$(\alpha_1, \alpha_2, \alpha_3, \alpha_4)$	ajustados empíricamente
Radio de inflado	4 celdas (0,20 m)
Lookahead distance	0,2 m
Velocidad lineal	0,1 m/s
Velocidad angular (ALIGNING)	0,3 rad/s
Tolerancia de posición	0,10 m
Tolerancia angular	12°
Umbral obstáculo (LIDAR)	0,20 m
CONE_GOAL_OFFSET	0,40 m
AREA_MINIMA_PX	300 px ²
Offset LIDAR TB4	90°

Cuadro 3: Hiperparámetros finales del sistema.

6. Diagnóstico Sim-to-Real

El traslado al hardware real introduce factores no modelados en las pruebas con bag.

Ruido y deriva de odometría: La odometría física acumula ruido notoriamente mayor que la simulada. Esto exige parámetros α_i más grandes y umbrales de degradación más generosos para que el filtro no sobre-confíe y pierda la pose. Estrategia: inflar los parámetros de ruido del modelo de movimiento.

Pérdida de tracción: El deslizamiento de ruedas rompe el supuesto del modelo de deltas y produce saltos de estimación que el filtro debe absorber. Los márgenes de inflado del mapa y las tolerancias de posición generosas mitigan este efecto.

Sensibilidad HSV a la iluminación: Los umbrales calibrados sobre el bag de visión pueden requerir reajuste ante cambios de luz del laboratorio o sombras. Se dejaron parametrizados y la imagen de *debug* permite ajustar in-situ.

Ruido del LIDAR real: Afecta tanto la fusión visión-LIDAR para estimar la distancia al cono como la localización del filtro. La ventana de ± 3 rayos más el mínimo, y el **CONE_GOAL_OFFSET** de 0,40 m, mitigan este efecto en la percepción.

7. Conclusiones

Se implementó un sistema autónomo completo sobre ROS 2 Humble y TurtleBot4:

1. **Parte A:** Graph SLAM con cierre de lazo por ID de ArUco sobre datos reales. Mapa coherente con el laberinto, apto para planificación y directamente reutilizable en el robot real.
2. **Parte B:** Navegación robusta con MCL, Θ^* y Pure Pursuit. La detección de obstáculos dinámicos evita falsos positivos con paredes conocidas.
3. **Parte C:** Cono rojo detectado con doble rango HSV fusionado con LIDAR. La separación entre percepción y planificación previene colisiones contra paredes caladas.

Bibliografía

- Thrun, S., Burgard, W., Fox, D. (2005). *Probabilistic Robotics*. MIT Press.
- Documentación ROS 2 Humble y OpenCV ArUco.